

UFT: Unifying Fine-Tuning of SFT and RLHF/DPO/UNA through a Generalized Implicit Reward Function

Zhichao Wang^{†*1}, Bin Bi^{†1}, Zixu Zhu¹, Xiangbo Mao¹, Jun Wang¹
 Shiyu Wang¹, Cheng Wan², Dong Nie³, Lingzi Hong⁴
¹Salesforce ²RadixArk ³ChatAlpha AI ⁴University of North Texas
 zcwang0201@gmail.com

May 11, 2026

Abstract

By pretraining on trillions of tokens, an LLM gains the capability of text generation. However, to enhance its utility and reduce potential harm, SFT and alignment are applied sequentially to the pretrained model. Because SFT and alignment have different objectives and underlying processes, performance on certain tasks can decline. To address this, we seamlessly introduce Unified Fine-Tuning (UFT), which integrates SFT and alignment into a single training stage using the same objective and loss functions through an implicit reward function. Our experimental results demonstrate that UFT outperforms SFT on instruction-tuning data alone. Moreover, when combining instruction-tuning data with alignment data, UFT effectively prevents the degradation on some tasks across these two stages and shows a clear advantage over sequentially applying SFT and alignment. This is evident in the significant improvements observed in the **ifeval** task for instruction-following and the **truthful** task for factuality. The proposed general fine-tuning framework UFT establishes an effective and efficient paradigm for LLM post-training.

1 Introduction

To enable large language models (LLMs) to understand and generate natural language, they are constructed with billions of parameters and pretrained on datasets containing trillions of tokens [OAA⁺24]. However, several challenges arise after the pretraining stage of LLMs [WBP⁺24]. One major issue is that pretrained LLMs can only continue generation based on the previous context and often struggle to accurately answer users’ questions. To address it, supervised fine-tuning (SFT) is introduced, using pairs of questions and answers. For example, in models like Mistral, preset instructions such as ‘[INST]’ and ‘[/INST]’ are used to frame a question as a prompt [JSM⁺23]. The corresponding answer is then used as the target output. The model’s probability of generating the correct answer is maximized through next-token prediction, employing the cross-entropy loss function to classify tokens across the entire token space.

The next challenge for LLMs lies in ethical concerns, where LLMs may inadvertently teach humans to engage in unethical activities, such as robbing banks [OWJ⁺22]. To address this issue, various alignment methodologies have been proposed, including Reinforcement Learning from Human Feedback (RLHF) [OWJ⁺22, BJN⁺22] with Proximal Policy Optimization (PPO) [SWD⁺17], Direct Preference Optimization (DPO) [RSM⁺23], Kahneman & Tversky Optimization (KTO) [EXM⁺24], and Unified Alignment (UNA) [WBH⁺24]. The core idea of alignment is to equip LLMs with the ability to reject harmful requests by learning from human feedback.

For pretrained LLMs, SFT and alignment are traditionally performed in sequence. However, this staged approach often leads to performance degradation, where the model loses capabilities acquired in earlier phases. This paper seeks to address and mitigate this degradation.

*: corresponding author; †: equal contribution
 github code: <https://github.com/zcw0201/UFT-UNA/new/main>

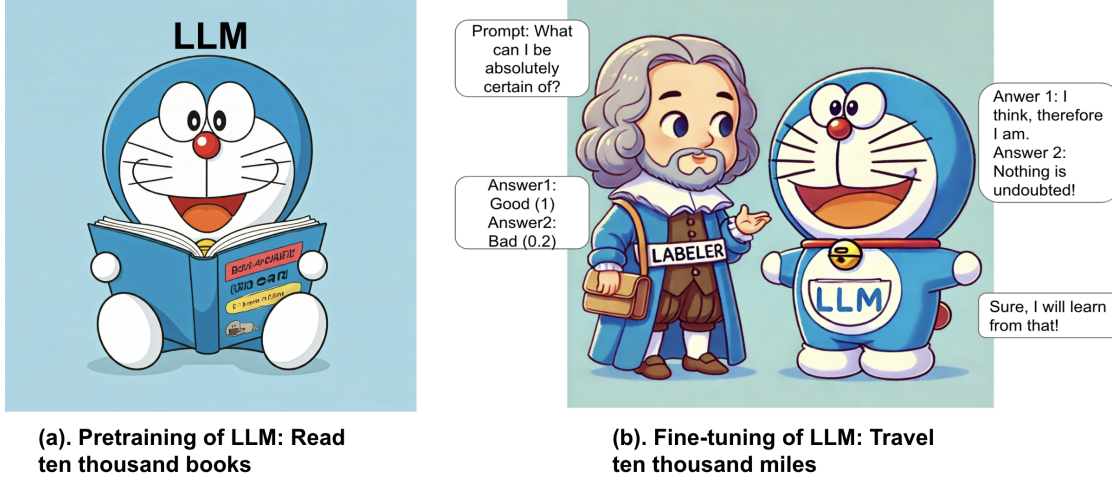


Figure 1: UFT integrates SFT and alignment through a generalized implicit reward function. It likens pre-training and fine-tuning of LLMs to Chinese proverb "Read ten thousand books, travel ten thousand miles". In pre-training, the LLM processes vast amounts of data without feedback, gaining broad language understanding. In fine-tuning, it generates responses to prompts and receives feedback, refining its abilities and improving performance on specific tasks.

Drawing inspiration from UNA’s effectiveness in handling score-based feedback, we propose extending UNA to also fulfill the goals of SFT. By converting SFT data into a format compatible with alignment training, we enable the use of a unified objective and loss function. This approach allows for effective fine-tuning of pretrained LLMs while preserving their previously acquired capabilities across tasks.

The contributions of this paper are listed as follows:

1. Prove that both UNA and SFT maximize the likelihood of the response in instruction-tuning data, and UNA outperforms SFT on downstream tasks when fine-tuning on instruction-tuning data.
2. UFT that unifies SFT and alignment solves the performance degradation on instruction-following and factuality tasks elegantly and surpasses the performance of the original sequential application order in downstream tasks.
3. UFT builds a unified post-training framework that is parallel to pretraining where the goal lies in generating responses for given prompts, receiving score-based feedback from different labelers and improve its capability on downstream tasks.

2 Methodology

In this section, we will explore the methodologies of SFT, RLHF, DPO, UNA, and the integrated framework UFT.

2.1 SFT

The pretrained LLM is limited to either continuing the text or repeating the question, which restricts its usefulness. To address these limitations and enhance the LLM’s question-answering capabilities, SFT is applied. The instruction-tuning dataset consists of numerous pairs of prompts (denoted as x) and responses (denoted as y). Given a prompt x , the probability of the pretrained LLM generating the response y is represented as $\pi_\theta(y|x)$. The objective of SFT is to maximize the probability of all response tokens by using cross-entropy loss as shown in Eq. 1 and Figure. 2(A).

$$L_{\text{SFT}}(\pi_\theta) = -\log(\pi_\theta(y|x)) \quad (1)$$

Suppose y is composed of N tokens, i.e., $y_1, y_2, \dots, y_N$. Based on Bayes' theorem, $\pi_\theta(y|x) = \prod_{i=1}^N \pi_\theta(y_i|x, y_1, \dots, y_{i-1})$. Applying log to Eq. 1, the SFT loss can be derived based on each token using cross entropy loss function on all candidate tokens for classification, i.e., $L_{\text{SFT}}(\pi_\theta) = -\sum_{i=1}^N \log(\pi_\theta(y_i|x, y_1, \dots, y_{i-1}))$.

2.2 RLHF, DPO and UNA

Even after the pretraining and SFT stages, LLMs can still produce undesired responses that may lead to bias or ethical issues. To address this, methods such as RLHF, DPO, and KTO have been proposed. A plot on the difference among RLHF, DPO and UNA can be found in Figure. 2(B). RLHF tackles these problems in two stages: reward model training and reinforcement learning. During the reward model training process, an explicit reward model is derived from pairwise data using the BT model, as illustrated in Eq. 2, where r_ϕ represents the explicit reward model. The dataset for training the reward model is composed of triplet of 1). prompt x , 2). desired response y_w and 3). undesired response y_l . The second stage of RLHF involves online reinforcement learning with the pretrained explicit reward model to generate reward signals. These signals are then combined with KL divergence to balance the reward and model capability obtained during the pretraining stage, as shown in Eq. 3. The RL process is optimized using PPO.

$$L_{\text{RM}}(\pi_\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log(\sigma(r_\phi(x, y_w) - r_\phi(x, y_l))) \right] \quad (2)$$

$$\begin{aligned} \pi_\theta^*(y|x) = \max_{\pi_\theta} \mathbb{E}_{x \sim D} \left[\mathbb{E}_{y \sim \pi_\theta(y|x)} [r_\phi(x, y)] \right. \\ \left. - \beta D_{\text{KL}}(\pi_\theta(y|x) \parallel \pi_{\text{ref}}(y|x)) \right] \end{aligned} \quad (3)$$

The training of RLHF is memory-intensive because it requires maintaining both the explicit reward model, the value model and the policy model. Additionally, reinforcement learning is notorious for its instability.

To address this issue, DPO proposes creating a mapping between the optimal policy and the reward model, i.e., an implicit reward model, as illustrated in Eq. 4, and optimizing them together. However, $Z(x)$ is intractable and can only be canceled out by subtracting the implicit reward of the desired response y_w , i.e., $r_\theta(x, y_w)$ from the implicit reward of the undesired response y_l , i.e., $r_\theta(x, y_l)$. This limitation confines DPO to pairwise datasets only. Furthermore, DPO cannot utilize the precise evaluation from the explicit reward model in RLHF. KTO extends DPO to binary feedback by estimating $Z(x)$ from multiple responses to the same prompt.

$$r_\theta(x, y) = \beta \log \left(\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right) + \beta \log Z(x) \quad (4)$$

To address this issue, UNA offers an alternative proof and demonstrates a novel mapping between the optimal policy and the reward model, i.e., the generalized implicit reward model. The form of the mapping between the generalized implicit reward model and policy is shown in Eq. 5. Unlike DPO, which is constrained by $Z(x)$ to pairwise feedback, UNA is versatile enough to handle various types of data, including pairwise feedback, binary feedback, and score-based feedback. These data types are optimized by minimizing the difference between the implicit reward in Eq. 5 and the explicit reward $r_\phi(x, y)$, which is provided by human labelers, other LLMs, or the explicit reward model, as shown in Eq. 6. When LLMs and explicit reward models are used to evaluate responses in real-time, it is referred to as online UNA. Conversely, if feedback is labeled beforehand, it is termed offline UNA. Consequently, UNA can function in both online and offline modes, effectively bridging the gap between online RLHF and offline DPO and KTO.

$$r_\theta(x, y) = \beta \log \left(\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right) \quad (5)$$

$$L_{\text{UNA}}(\pi_\theta) = \mathbb{E}_{x \sim D, y \sim \pi_\theta(\cdot|x)} [g(r_\phi(x, y), r_\theta(x, y))] \quad (6)$$

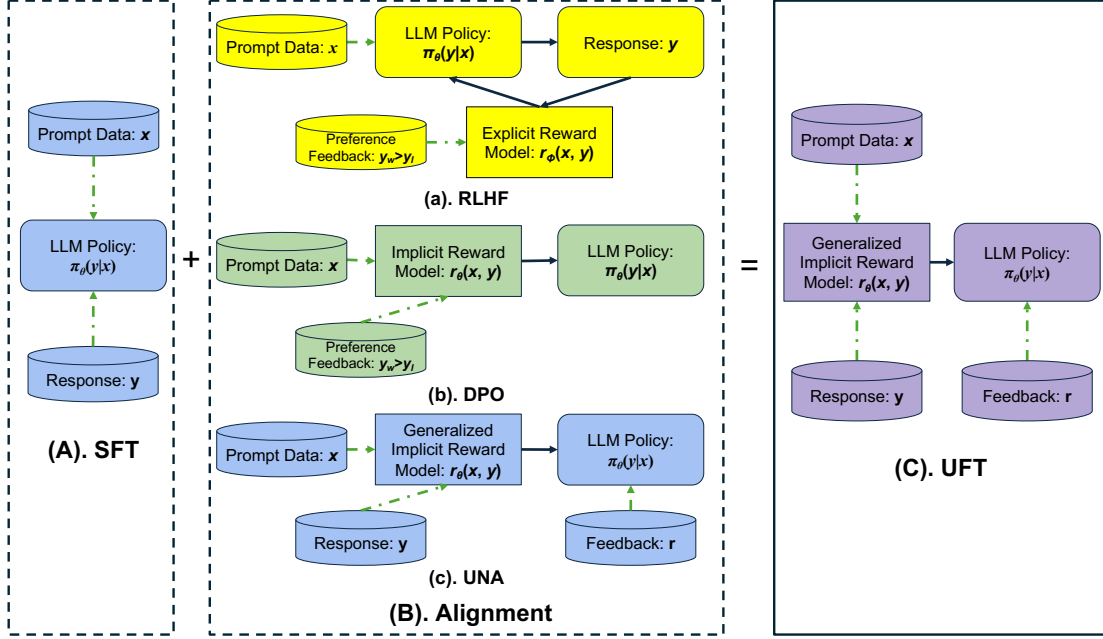


Figure 2: (A) refers to SFT, (B) refers to alignment including RLHF, DPO and UNA and (C) refers to UFT. Traditionally, the fine-tuning process begins with SFT followed by alignment. However, the proposed UFT method integrates both SFT and alignment into a single, cohesive process.

2.3 UFT: Unify SFT and Alignment

To enhance the capabilities of LLMs in question answering and addressing ethical issues, SFT and alignment are typically applied in a sequential manner due to the distinct nature of these tasks. However, this sequential approach often leads to performance degradation on some tasks. In this study, we introduce UFT which integrates SFT and alignment into a single stage, thereby mitigating this problem.

To be more specific, the instruction-tuning data consist of a prompt x and a corresponding response y , where the response y is considered of high quality, typically labeled by experts in the field. In comparison, the alignment data include a prompt x , response y , and feedback r . This feedback can be categorized as desired or undesired for pairwise feedback, positive or negative for binary feedback, or a scalar in the interval $[0, 1]$ for general score-based feedback. Due to the high quality of instruction-tuning data, they can be regarded as data with a score of 1, i.e., positive feedback. With this consideration, the instruction-tuning dataset can be transformed into alignment data in the format of prompt x , response y and feedback $r = 1$, which is the highest reward in consideration. Eventually, the transformed instruction-tuning dataset and the alignment dataset can be merged for fine-tuning LLM using the loss function in UNA as shown in Eq. 6. Our experiments demonstrate that when fine-tuning using only instruction-tuning dataset, UFT can outperform SFT on downstream tasks, which we attribute to the KL divergence term that focuses on minimization with the pretrained model, a factor ignored in the SFT processes. Additionally, the results indicate that mixing the instruction-tuning data with alignment for UFT can prevent the performance degradation on some tasks and outperform previous sequential methods. Lastly, we discover that the distribution of mixed data, i.e., the proportion of instruction-tuning data and alignment data will impact the performances of LLMs. More details can be found in the experiment section.

A heuristic proof why UFT can replace SFT is provided by arguing that they achieve the same goal of maximizing the probability of $\pi_\theta(y|x)$ for prompt x and response y in the instruction-tuning dataset. For SFT, the probability of $\pi_\theta(y|x)$ is directly maximized by cross entropy-loss. In UFT, suppose we apply the Sigmoid function on the implicit reward in Eq. 5 and using mean square error (MSE) to measure the difference of implicit reward and explicit reward, the objective will become Eq. 7. This loss function will drive $r_\theta(x, y) = \beta \log \left(\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right)$ to positive infinity. Since both β and $\pi_{\text{ref}}(y|x)$ are fixed, this objective will maximize $\pi_\theta(y|x)$. As a result, UFT can replace SFT to maximize

the probability of generating the response, and it outperforms SFT by minimizing the difference to the pretrained model.

$$\begin{aligned} L_{\text{UFT-SFT}}(\pi_\theta) &= \mathbb{E}_{(x,y) \sim D} \left[[\sigma(r_\theta(x, y)) - 1]^2 \right] \\ &= \mathbb{E}_{(x,y) \sim D} \left\{ \left[\sigma \left(\beta \log \left(\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right) \right) - 1 \right]^2 \right\} \end{aligned} \quad (7)$$

The generalized UFT loss function is presented in Equation 8. For a given tuple comprising a prompt x , response y , and explicit reward score $r_\phi(x, y)$, the implicit reward $r_\theta(x, y)$ is first calculated using Equation 5. Subsequently, the difference between the implicit and explicit reward scores is minimized through a general function $g(\cdot)$. Specifically, for Supervised Fine-Tuning (SFT) data, the explicit reward score $r_\phi(x, y)$ is set to its maximum value. This approach enables a unified framework for the post-training processes of SFT and alignment.

$$L_{\text{UFT}}(\pi_\theta) = \mathbb{E}_{(x,y) \sim D} \left[g(r_\theta(x, y), r_\phi(x, y)) \right] \quad (8)$$

In summary, UFT reformats the data structure of instruction-tuning data to ensure compatibility with UNA, thereby enabling the UFT framework to unify SFT and alignment processes. An offline version of UFT is illustrated in part (C) of Figure 2. This offline UFT can be transitioned to an online version, provided that the pre-collected feedback is gathered in real-time using a reward model or other LLMs.

2.4 Comparison of Pretraining and Fine-Tuning

By integrating SFT and alignment through UFT, we can establish a unified fine-tuning framework that runs parallel to the pretraining phase. Drawing inspiration from the old Chinese proverb "Read ten thousand books, travel ten thousand miles", we can liken the pretraining stage to "Read ten thousand books". In this phase, a LLM absorbs and learns from an extensive collection of tokens, amounting to trillions. The primary objective during this stage is to predict the next token based on the previous context window, optimizing the model's performance through the cross entropy loss function. Conversely, the fine-tuning stage can be likened to the adage "Travel ten thousand miles," where the model is exposed to a wide array of prompts. It generates diverse responses and learns from the feedback it receives, thereby refining and enhancing its capabilities.

3 Experiments

In this section, a series of experiments will be conducted to demonstrate the advantages of UFT across various use cases. Initially, a comparison between UFT and SFT on instruction-tuning data will be performed to illustrate that UFT surpasses SFT. Following this, the second experiment will compare the performance of UFT when applied to SFT and alignment data simultaneously versus applying SFT and alignment sequentially. The third experiment will examine the impact of data distribution between SFT and alignment data, which is crucial for the outcomes of fine-tuning.

4 Experiment Setup

To begin with, we compare UFT with SFT on the same instruction-tuning dataset. The UltraChat dataset is utilized by unfolding one conversation into multiple training examples to facilitate the training of UFT. From the UltraChat dataset, 20k samples are selected and utilized for training. The Mistral 7B-v0.1 [JSM⁺23] and Qwen 32B [QY⁺25] are utilized as the base model, with low rank adaptation (LoRA) of $r = 16$ [HSW⁺21]. For SFT, the learning rate of $1e^{-4}$ is the best, and for UFT, the learning rate of $3e^{-5}$ and β of 0.01 is the best.

In UFT vs SFT+Alignment, we utilize the HelpSteer2 dataset of 20k examples for alignment [WDD⁺24]. For UFT, the 20k examples from UltraChat and 20k examples from HelpSteer2 are merged and utilized for training. For comparison, the best performing SFT model of learning rate $1e^{-4}$ in the previous experiment is utilized for further fine-tuning using DPO, KTO and UNA.

Model	bbh	gpqa	mmlu-pro	musr	ifeval	math-hard	average
Mistral	44.11	29.53	30.11	41.79	23.22	2.92	28.61
Mistral+SFT	46.04	28.72	29.35	42.94	29.5	2.66	29.87
Mistral+UFT	46.55	29.24	30.25	41.73	28.89	3.87	30.09
Mistral+SFT+DPO	44.52	29.98	29.95	40.31	26.64	3.13	29.09
Mistral+SFT+KTO	42.89	31	30.48	40.59	25.17	2.94	28.85
Mistral+SFT+UNA	43.74	30.78	30.09	40.56	26.82	2.96	29.16
Mistral+UFT	45.46	31.15	30.05	41.06	46.03	3.13	32.81

Table 1: Comparison of Mistral+SFT and Mistral+UFT on instruction-tuning data, and comparison of Mistral+SFT+DPO, Mistral+SFT+KTO, Mistral+SFT+UNA, and Mistral+UFT on both instruction-tuning and alignment data on the new HuggingFace open LLM Leaderboard

Model	gsm8k	truthful	winograde	arc	hellaswag	mmlu	average
Mistral	38.02	42.58	77.58	61.43	83.44	62.51	60.93
Mistral+SFT	39.65	51.06	78.53	63.99	83.78	61.99	63.17
Mistral+UFT	45.57	51.18	78.93	63.82	83.54	62.44	64.25
Mistral+SFT+DPO	42.19	47.83	78.45	62.16	84.03	62.38	62.84
Mistral+SFT+KTO	42.57	49.67	79.4	61.86	83.83	62.06	63.23
Mistral+SFT+UNA	39.99	49.54	79.72	62.46	84.08	62.3	63.02
Mistral+UFT	41.59	54.05	79.79	63.82	84.44	62.33	64.34

Table 2: Comparison of Mistral+SFT and Mistral+UFT on instruction-tuning data, and comparison of Mistral+SFT+DPO, Mistral+SFT+KTO, Mistral+SFT+UNA, and Mistral+UFT on both instruction-tuning and alignment data on the old HuggingFace open LLM Leaderboard

4.1 UFT vs. SFT

The fine-tuned models are tested on different tasks including 12 tasks on two HuggingFace Open LLM Leaderboards[BFH⁺23, FHL⁺24]. The new Open LLM Leaderboard includes 6 tasks: **bbh** (Suzgun et al., 2022), **gpqa** (Rein et al., 2023), **mmlu-pro** (Wang et al., 2024), **musr** (Sprague et al., 2024), **ifeval** (Zhou et al., 2023), and **math-hard** (Hendrycks et al., 2021). For all these tasks, the average scores are reported. Conversely, the old Open LLM Leaderboard comprises 6 different tasks: **gsm8k** (Cobbe et al., 2021), **truthful** (Lin et al., 2022), **winograde** (Sakaguchi et al., 2019), **arc** (Allen AI), **hellaswag** (Zellers et al., 2019), and **mmlu** (Hendrycks et al., 2021). In this work, the average match rate in **gsm8k**, mc2 in **truthful**, accuracy in **winograde**, acc-norm in **arc**, acc-norm in **hellaswag**, and accuracy in **mmlu** will be reported. Additionally, MT-Bench [ZCS⁺23] and Alpaca-eval [LZD⁺23] will be used to evaluate the model’s ability to generate text responses, rather than selecting from predefined candidate answers. Specifically, the Length-Controlled Win Rate (LC WR) will be documented for Alpaca-eval. Additionally, the average length of the outputs within this evaluation will be provided for comprehensive analysis.

As demonstrated in Table 1 and Table 2, Mistral+UFT surpasses Mistral+SFT in 8 out of 12 tasks.

Model	MT-Bench	Alpaca-eval	
		LC WR	Length
Mistral	3.15	0.31	6554
Mistral+SFT	6.33	8.07	908
Mistral+UFT	6.55	7.27	974
Mistral+SFT+DPO	4.81	1.05	5654
Mistral+SFT+KTO	4.76	0.64	6215
Mistral+SFT+UNA	5.24	1.34	4945
Mistral+UFT	6.78	8.28	1317

Table 3: Comparison of Mistral+SFT and Mistral+UFT on instruction-tuning data, and comparison of Mistral+SFT+DPO, Mistral+SFT+KTO, Mistral+SFT+UNA, and Mistral+UFT on both instruction-tuning and alignment data using MT-Bench and Alpaca-eval

Model	bbh	gpqa	mmlu-pro	musr	ifeval	math-hard	average
Qwen	67.89	37.32	57.93	49.12	42.98	31.63	47.81
Qwen+SFT	67.75	40.42	58.13	48.73	47.48	30.77	48.88
Qwen+UFT	67.67	40.42	58.1	48.47	46.36	31.23	48.71
Qwen+SFT+DPO	67.8	39.01	58.06	49.4	47.48	33.67	49.24
Mistral+SFT+KTO	67.61	39.26	58.19	48.34	53.03	35.07	50.27
Mistral+SFT+UNA	68.84	38.42	58.35	49.14	57.33	34.7	51.13
Mistral+UFT	67.73	38.59	58.73	47.42	64.05	37.8	52.39

Table 4: Comparison of Qwen+SFT and Qwen+UFT on instruction-tuning data, and comparison of Qwen+SFT+DPO, Qwen+SFT+KTO, Qwen+SFT+UNA, and Qwen+UFT on both instruction-tuning and alignment data on the new HuggingFace open LLM Leaderboard

Model	gsm8k	truthful	winograde	arc	hellaswag	mmlu	average
Qwen	85.79	57.75	82	70.31	85.22	83.19	77.38
Qwen+SFT	87.38	58.81	82	70.9	84.99	83.39	77.91
Qwen+UFT	88.29	58.96	82.64	70.9	85	83.34	78.19
Qwen+SFT+DPO	85.67	58.54	82	70.14	85.24	83.31	77.48
Qwen+SFT+KTO	85.56	59.77	82.16	70.14	85.32	83.32	77.71
Qwen+SFT+UNA	92.08	62.74	82.4	71.5	85.37	83.36	79.58
Qwen+UFT	90.68	66.7	83.58	71.84	85.58	83.35	80.29

Table 5: Comparison of Qwen+SFT and Qwen+UFT on instruction-tuning data, and comparison of Qwen+SFT+DPO, Qwen+SFT+KTO, Qwen+SFT+UNA, and Qwen+UFT on both instruction-tuning and alignment data on the old HuggingFace open LLM Leaderboard

When considering the average performances across these leaderboards, Mistral+UFT consistently outperforms Mistral+SFT. This indicates that UFT enhances the capabilities of an LLM by maximizing the reward and minimizing the difference from the pretrained model. Furthermore, in the MT-Bench evaluation, Mistral+UFT outperforms Mistral+SFT, whereas in the Alpaca-eval, Mistral+SFT overshadows Mistral+UFT. From a generation capability standpoint, both UFT and SFT exhibit similar performance. Similar conclusion can be found for Qwen 32B. Qwen+UFT outperforms Qwen+SFT in 10 out of 14 tasks as shown in Table 4 and Table 5 and both MT-Bench and Alpaca-eval in Table 6. As a result, when evaluating the performance on downstream tasks, UFT demonstrates superiority over SFT.

4.2 UFT vs SFT+Alignment

The same tasks are utilized for evaluation. In the two HuggingFace open LLM leaderboards, Mistral+UFT outperforms all of Mistral+SFT+DPO, Mistral+SFT+KTO, and Mistral+SFT+UNA in 9 out of 12 tasks. In terms of average scores, Mistral+UFT surpasses all three sequential methods. Several aspects need further discussion. Firstly, the performance degradation problem is evident as the

Model	MT-Bench	Alpaca-eval	
		LC WR	Length
Qwen	7.97	8.47	2176
Qwen+SFT	7.85	8.34	2800
Qwen+UFT	7.95	9.96	1292
Qwen+SFT+DPO	8.64	9.83	2664
Qwen+SFT+KTO	8.58	10.17	2149
Qwen+SFT+UNA	8.57	13.75	1328
Qwen+UFT	8.67	13.79	1307

Table 6: Comparison of Qwen+SFT and Qwen+UFT on instruction-tuning data, and comparison of Qwen+SFT+DPO, Qwen+SFT+KTO, Qwen+SFT+UNA, and Qwen+UFT on both instruction-tuning and alignment data using MT-Bench and Alpaca-eval

Model	bbh	gpqa	mmlu-pro	musr	ifeval	math-hard	average
Mistral	44.11	29.53	30.11	41.79	23.22	2.92	28.61
UNA(16k+20k)	45.17	30.64	29.95	39.19	44.21	2.38	31.92
UNA(20k+20k)	45.46	31.15	30.05	41.06	46.03	3.13	32.81
UNA(32k+20k)	44.75	31.28	29.76	39.06	46.76	3.51	32.52
UNA(65k+20k)	44.4	31.38	29.66	36.8	41.91	3.51	31.28
UNA(130k+20k)	44.35	29.75	30.02	38.66	44.2	2.99	31.66
UNA(260k+20k)	44.31	31	30.1	39.85	45.8	3.26	32.39

Table 7: Impact of different distributions of instruction-tuning and alignment data on the new HuggingFace Open LLM Leaderboard

Model	gsm8k	truthful	winograde	arc	hellaswag	mmlu	average
Mistral	38.02	42.58	77.58	61.43	83.44	62.51	60.93
UNA(16k+20k)	40.83	56.69	79.4	64.85	84.66	62.22	64.78
UNA(20k+20k)	41.59	54.05	79.79	63.82	84.44	62.33	64.34
UNA(32k+20k)	40.83	54.38	79.72	64.42	84.46	62.88	64.45
UNA(65k+20k)	41.43	52.35	79.79	65.27	84.19	61.89	64.15
UNA(130k+20k)	41.13	50.17	79.48	64.59	84.07	62.26	63.62
UNA(260k+20k)	41.21	50.17	80.43	64.85	83.89	62.35	63.82

Table 8: Impact of different distributions of instruction-tuning and alignment data on the old HuggingFace Open LLM Leaderboard

performances of Mistral+SFT+DPO, Mistral+SFT+KTO, and Mistral+SFT+UNA are worse than Mistral+SFT, a phenomenon also known as alignment tax. Additionally, we observe that Mistral+UFT shows significant improvements on **ifeval**, which tests the model’s capability of instruction-following, and **truthful**, which assesses the model’s alignment capabilities. These results indicate that UFT greatly enhances instruction-following and alignment capabilities, demonstrating its effectiveness. In terms of generation capability, Mistral+UFT outperforms the other three sequential methods and it does not seem to suffer from performance degradation, as it performs better than both Mistral+SFT and Mistral+UFT on instruction-tuning data alone. Moreover, Mistral+UFT does not bias towards long generation like the other three sequential methods, which is another advantage of UFT. Similar observations can be found for Qwen 32B. Qwen+UFT outperforms Qwen+SFT+Alignment in 9 out of 14 tasks, especially the average of the new and old HuggingFace open LLM leaderboards as shown in Table. 4 and Table. 5. For generation capability, Qwen+UFT outperforms Qwen+SFT+Alignment on both MT-Bench and Alpaca-eval, though the benefits diminish compared with smaller Mistral 7B model. This means that the performance degradation caused by SFT and alignment is diminished with larger model size, and this is consistent the conclusions of previous works on RLHF [BJN⁺22].

4.3 Data Distribution in Training UFT

During the pretraining phase, ensuring a balanced data distribution is crucial for optimizing the capabilities of a LLM. In line with this principle, we have incorporated varying percentages of the

Model	MT-Bench	Alpaca-eval	
		LC WR	Length
Mistral	3.15	0.31	6554
UNA(16k+20k)	6.25	7.9	1378
UNA(20k+20k)	6.78	8.28	1317
UNA(32k+20k)	6.54	8.23	1324
UNA(65k+20k)	6.3	9.92	1338
UNA(130k+20k)	6.83	7.43	1361
UNA(260k+20k)	6.45	6.85	1378

Table 9: Impact of different distributions of instruction-tuning and alignment data on MT-Bench and Alpaca-eval

instruction-tuning data into the alignment dataset to create a final training dataset. Specifically, we have kept the alignment dataset constant, i.e., 20k while integrating 260k, 130k, 65k, 32k, and 16k samples from the UltraChat dataset. These combined datasets are then used for fine-tuning, allowing us to observe the impact of different data distributions on the model’s performance.

Several intriguing points will be discussed. As shown in Table 7 and Table 8, among the 12 tasks, one task, namely **musr**, shows a minor statistical decrease in performance. In contrast, four tasks—**gpqa**, **gsm8k**, **winograde**, and **arc**—exhibit statistically small improvements. Notably, two tasks, **ifeval** and **truthful**, demonstrate statistically significant improvements. Specifically, **ifeval** improves from 23.22 to around 44, and **truthful** improves from 42.58 to around 53. Significant advancements are observed in MT-Bench and Alpaca-eval, indicating a substantial enhancement in the generation capability of the LLM as shown in Table 9. This aligns with our expectations, as instruction-tuning data primarily aim to enhance the model’s instruction-following capabilities, which positively impacts **ifeval**, MT-Bench, and Alpaca-eval. Meanwhile, alignment data contribute to improvements in tasks like **truthful**. Other tasks remain largely unaffected due to the absence of relevant data, suggesting that incorporating more pertinent data could enhance their performance. When increasing the proportion of instruction-tuning data, we observe a decrease in **truthful** performance from 56 to 50, indicating that a larger proportion of alignment data benefits the bias and ethical performance of the LLM. On the other hand, simply adding more instruction-tuning data does not improve the performance of **ifeval**, MT-Bench, and Alpaca-eval. Therefore, further investigation into the optimal ratio between instruction-tuning data and alignment data is warranted.

5 Related Work

The field of large language models (LLMs) has undergone significant advancements, marked by models trained on trillions of tokens and comprising billions of parameters, often processed in parallel during their pretraining phase [OAA⁺24, Ant24, TAB⁺23]. Subsequently, supervised fine-tuning (SFT) becomes essential for enhancing the model’s performance across diverse downstream tasks.

However, neither pretraining nor SFT can fully address the issues of bias and ethics in LLMs [OAA⁺24, WBP⁺24]. To tackle these challenges, RLHF with PPO has been proposed and is widely accepted for aligning LLMs, including GPT and Claude [OWJ⁺22, BJN⁺22]. Despite its popularity, RLHF/PPO faces several issues, such as high memory requirements, instability in reinforcement learning, and the need for multiple training stages, including reward model (RM) training and RL fine-tuning [RSM⁺23]. To reduce the cost of human labeling, AI feedback can be used to replace human feedback, a method known as reinforcement learning from AI feedback (RLAIF) [BKK⁺22, LPM⁺23]. RLOO argues that PPO is excessive for LLM alignment since the model has already been pretrained, suggesting that RLOO should suffice [ACG⁺24].

To simplify RLHF, DPO has been proposed to map the optimal policy and reward model into a single step, transforming the initial unstable RL into a binary cross-entropy problem [RSM⁺23]. Several downstream studies have been conducted on DPO to expand and generalize its capabilities, such as different loss functions [PKD⁺24, ARP⁺23], iterative approaches [YPC⁺24, XLSW24], token DPO [RHPF24, ZLM⁺24] and stepwise DPO [KKS⁺24].

Previous work focused on pairwise datasets, which are more challenging to gather. In contrast, binary feedback like "thumbs up" and "thumbs down" is easier to collect. KTO leverages the concept of human aversion to undesired data and successfully handles binary feedback [EXM⁺24]. DRO focuses on binary data by estimating the policy and value functions and optimizing each sequentially while keeping the other fixed [RTG⁺24]. However, these methods cannot accommodate different types of data. To address this, UNA was proposed to handle pairwise, binary, and score-based feedback through an implicit reward model, supporting both online and offline alignment [WBH⁺24].

There have also been efforts to merge SFT with alignment. ORPO proposed a new loss function to increase the ratio of desired responses over undesired responses, achieving both SFT and alignment [HLT24]. PAFT suggested conducting SFT and alignment in parallel and merging them afterward [PWB⁺24]. However, ORPO’s reliance on pairwise datasets and its deteriorating performance compared to other SFT and alignment methods pose challenges. In contrast, PAFT requires training separate adaptors for SFT and alignment and merging them through sparsity, which is inefficient. Inspired by UNA’s achievements, we aim to unify SFT with alignment based on UNA’s principles to avoid performance degradation.

6 Conclusion

During pretraining, an LLM is trained on vast datasets to predict the next token based on a given context window, enabling it to learn rich linguistic patterns and general knowledge. In contrast, fine-tuning involves exposing the model to specific prompts, generating responses, receiving feedback, and refining its performance on targeted downstream tasks. However, traditional sequential approaches to SFT and alignment can disrupt the model’s pretrained knowledge, leading to a decline in generalization or task-specific performance.

UFT addresses these challenges by streamlining the fine-tuning process. When trained exclusively on instruction-tuning data, UFT outperforms conventional SFT by minimizing divergence from the pretrained model, achieved through careful optimization of KL divergence. This ensures that the model retains its foundational capabilities while adapting to new tasks. Furthermore, by strategically blending instruction-tuning and alignment data, UFT surpasses traditional sequential SFT+alignment methods, effectively mitigating performance degradation across a range of tasks. In summary, UFT establishes a powerful and efficient fine-tuning paradigm that balances task-specific adaptation with the retention of pretrained knowledge, paving the way for more capable and versatile LLMs.

7 Limitation

This study presents two primary limitations that warrant attention in future research. First, the analysis is restricted to English, necessitating a comprehensive evaluation of the approach’s multilingual capabilities. Second, the current research primarily utilized academic datasets (e.g., Ultrachat and Helpsteer). To enhance the real-world applicability of these findings, further validation on diverse industrial datasets is recommended.

References

- [ACG⁺24] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms, 2024.
- [Ant24] AI Anthropic. The claude 3 model family: Opus, sonnet, haiku. *Claude-3 Model Card*, 1, 2024.
- [ARP⁺23] Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences, 2023.
- [BFH⁺23] Edward Beeching, Clémentine Fourrier, Nathan Habib, Sheon Han, Nathan Lambert, Nazneen Rajani, Omar Sanseviero, Lewis Tunstall, and Thomas Wolf. Open llm leaderboard (2023-2024). https://huggingface.co/spaces/open-llm-leaderboard-old/open_llm_leaderboard, 2023.
- [BJN⁺22] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. Training a helpful and harmless assistant with reinforcement learning from human feedback, 2022.
- [BKK⁺22] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin

- Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional ai: Harmlessness from ai feedback, 2022.
- [EXM⁺24] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. Kto: Model alignment as prospect theoretic optimization, 2024.
- [FHL⁺24] Clémentine Fourier, Nathan Habib, Alina Lozovskaya, Konrad Szafer, and Thomas Wolf. Open llm leaderboard v2. https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard, 2024.
- [HLT24] Jiwoo Hong, Noah Lee, and James Thorne. Orpo: Monolithic preference optimization without reference model, 2024.
- [HSW⁺21] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [JSM⁺23] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023.
- [KKS⁺24] Dahyun Kim, Yungi Kim, Wonho Song, Hyeonwoo Kim, Yunsu Kim, Sanghoon Kim, and Chanjun Park. sdpo: Don’t use your data all at once, 2024.
- [LPM⁺23] Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, and Sushant Prakash. Rlaif: Scaling reinforcement learning from human feedback with ai feedback, 2023.
- [LZD⁺23] Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. AlpacaEval: An automatic evaluator of instruction-following models. https://github.com/tatsu-lab/alpaca_eval, 2023.
- [OAA⁺24] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Lukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Lukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin,

Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Felipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report, 2024.

- [OWJ⁺22] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.
- [PKD⁺24] Arka Pal, Deep Karkhanis, Samuel Dooley, Manley Roberts, Siddhartha Naidu, and Colin White. Smaug: Fixing failure modes of preference optimisation with dpo-positive, 2024.
- [PWB⁺24] Shiva Kumar Pentiyala, Zhichao Wang, Bin Bi, Kiran Ramnath, Xiang-Bo Mao, Regunathan Radhakrishnan, Sitaram Asur, Na, and Cheng. Paft: A parallel training paradigm for effective llm fine-tuning, 2024.
- [QY⁺25] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025.
- [RHPF24] Rafael Rafailov, Joey Hejna, Ryan Park, and Chelsea Finn. From r to q^* : Your language model is secretly a q -function, 2024.
- [RSM⁺23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023.
- [RTG⁺24] Pierre Harvey Richemond, Yunhao Tang, Daniel Guo, Daniele Calandriello, Mohammad Gheshlaghi Azar, Rafael Rafailov, Bernardo Avila Pires, Eugene Tarassov, Lucas Spangher, Will Ellsworth, Aliaksei Severyn, Jonathan Mallinson, Lior Shani, Gil Shamir, Rishabh Joshi, Tianqi Liu, Remi Munos, and Bilal Piot. Offline regularised reinforcement learning for large language models alignment, 2024.

- [SWD⁺17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- [TAB⁺23] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- [WBH⁺24] Zhichao Wang, Bin Bi, Can Huang, Shiva Kumar Pentya, Zixu James Zhu, Sitaram Asur, and Na Claire Cheng. Una: Unifying alignments of rlhf/ppo, dpo and kto by a generalized implicit reward function, 2024.
- [WBP⁺24] Zhichao Wang, Bin Bi, Shiva Kumar Pentya, Kiran Ramnath, Sougata Chaudhuri, Shubham Mehrotra, Zixu, Zhu, Xiang-Bo Mao, Sitaram Asur, Na, and Cheng. A comprehensive survey of llm alignment techniques: Rlhf, rlaif, ppo, dpo and more, 2024.
- [WDD⁺24] Zhilin Wang, Yi Dong, Olivier Delalleau, Jiaqi Zeng, Gerald Shen, Daniel Egert, Jimmy J. Zhang, Makesh Narsimhan Sreedhar, and Oleksii Kuchaiev. Helpsteer2: Open-source dataset for training top-performing reward models, 2024.
- [XLSW24] Jing Xu, Andrew Lee, Sainbayar Sukhbaatar, and Jason Weston. Some things are more cringe than others: Iterative preference optimization with the pairwise cringe loss, 2024.
- [YPC⁺24] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models, 2024.
- [ZCS⁺23] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.
- [ZLM⁺24] Yongcheng Zeng, Guoqing Liu, Weiyu Ma, Ning Yang, Haifeng Zhang, and Jun Wang. Token-level direct preference optimization, 2024.

LR	bbh	gpqa	mmlu-pro	musr	ifeval	math-hard	average
$3e^{-6}$	45.87	29.39	30.21	42.41	25.2	3.61	29.45
$1e^{-5}$	47.14	29.59	30.21	41.74	25.77	3.12	29.6
$3e^{-5}$	47.71	29.76	30.1	40.95	26.96	3.01	29.75
$1e^{-4}$	46.04	28.72	29.35	42.94	29.5	2.66	29.87
$3e^{-4}$	42.79	28.17	25.22	43.61	19.78	2.3	26.98

Table 10: The impact of different learning rates for SFT on the new HuggingFace Open LLM Leaderboard

LR	gsm8k	truthful	winograde	arc	hellaswag	mmlu	average
$3e^{-6}$	42.95	47.9	79.32	61.86	83.34	62.37	62.96
$1e^{-5}$	43.25	49.58	79.08	62.29	83.33	62.15	63.28
$3e^{-5}$	44.47	50.5	78.93	62.54	83.25	62.61	63.72
$1e^{-4}$	39.65	51.06	78.53	63.99	83.78	61.99	63.17
$3e^{-4}$	13.39	46.43	75.85	58.53	81.16	55.73	55.18

Table 11: The impact of different learning rates for SFT on the old HuggingFace Open LLM Leaderboard

A SFT on 20k examples with Different LR

The impact of different learning rates on the performances of SFT can be found in Table. 10 and Table. 11 for the new and old HuggingFace Open LLM Leaderboards.

B UFT on 20k examples with Different LR and β

The impact of different learning rates and β on the performances of UFT can be found in Table. 12 and Table. 13 for the new and old HuggingFace Open LLM Leaderboards.

LR/ β	bbh	gpqa	mmlu-pro	musr	ifeval	math-hard	average
$1e^{-4}/0.01$	47.01	27.86	30.3	39.07	32.49	3.57	30.08
$1e^{-4}/0.03$	46.09	29.5	30.09	40.8	29.27	3.84	29.93
$1e^{-4}/0.1$	45.57	30.5	30.15	42.42	27.21	2.94	29.8
$1e^{-4}/0.3$	44.02	30.04	29.65	43.76	26.08	2.64	29.37
$3e^{-5}/0.01$	46.55	29.24	30.25	41.73	28.89	3.87	30.09
$3e^{-5}/0.03$	46.56	28.72	30.44	40.67	29.7	3.08	29.86
$3e^{-5}/0.1$	46.31	29.29	30.7	40.95	26.52	3.5	29.55
$3e^{-5}/0.3$	44.91	28.6	30.44	43.1	26.54	3.1	29.45
$1e^{-5}/0.01$	46.44	29.27	30.44	38.15	26.02	3.59	28.99
$1e^{-5}/0.03$	46.46	29.51	30.54	39.21	25.39	3.02	29.02
$1e^{-5}/0.1$	45.8	29.99	30.41	42.96	26.67	3.01	29.81
$1e^{-5}/0.3$	45.03	29.74	30.46	42.97	25	2.57	29.3
$3e^{-6}/0.01$	45.83	29.86	30.5	41.5	25.42	2.8	29.32
$3e^{-6}/0.03$	45.81	29.91	30.45	41.24	25.23	2.58	29.2
$3e^{-6}/0.1$	45.48	30.61	30.53	43.09	23.87	3.27	29.48
$3e^{-6}/0.3$	44.3	29.48	30.29	42.71	22.56	3.06	28.73

Table 12: The impact of different learning rates for SFT on the new HuggingFace Open LLM Leader-board

LR/ β	gsm8k	truthful	winograde	arc	hellaswag	mmlu	average
$1e^{-4}/0.01$	43.64	49.77	78.77	63.91	83.88	62.31	63.71
$1e^{-4}/0.03$	42.57	50.2	78.61	64.25	83.55	61.87	63.51
$1e^{-4}/0.1$	40.11	47.95	77.98	63.4	83.37	62.19	62.5
$1e^{-4}/0.3$	35.94	44.4	78.93	62.2	82.92	62	61.07
$3e^{-5}/0.01$	45.57	51.18	78.93	63.82	83.54	62.44	64.25
$3e^{-5}/0.03$	43.33	51.24	78.61	64.08	83.41	62.61	63.88
$3e^{-5}/0.1$	41.55	49.25	78.69	62.54	83.31	62.44	62.96
$3e^{-5}/0.3$	38.44	43.94	78.14	61.77	83.5	62.31	61.35
$1e^{-5}/0.01$	42.46	50.41	78.69	63.14	83.27	62.43	63.4
$1e^{-5}/0.03$	41.58	50.35	79.01	62.71	83.35	62.59	63.27
$1e^{-5}/0.1$	41.17	48.63	78.69	62.54	83.38	62.35	62.79
$1e^{-5}/0.3$	39.24	44.24	78.45	62.2	83.55	62.5	61.7
$3e^{-6}/0.01$	39.92	48.42	78.53	62.2	83.32	62.35	62.46
$3e^{-6}/0.03$	40.41	48.08	78.69	62.2	83.34	62.5	62.54
$3e^{-6}/0.1$	38.63	46.3	78.22	61.95	83.39	62.58	61.85
$3e^{-6}/0.3$	38.86	43.99	77.9	61.52	83.48	62.59	61.39

Table 13: The impact of different learning rates for SFT on the old HuggingFace Open LLM Leader-board